

Discovery Executive Summary

Project: discovery-12 · Generated: 10/07/2026, 13:37:12

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Code Quality & Complexity Analysis	HIGH RISK	82 / 100 — High Risk

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Classes (H7), Missing Frontend Service Layer (F2), and Business Logic in Components (F1).

EXECUTIVE SUMMARY

The multi-agent-web-ui codebase demonstrates a microservices architecture with React frontend and Node.js services, but suffers from significant architectural debt. The most critical issues are oversized god classes (3171 LOC useAppStore, 1414 LOC AgentDetail component), missing service layer abstraction with business logic embedded directly in React components, and weak domain boundaries between frontend presentation and backend API layers. While the microservices pattern provides good separation at the service level, the frontend lacks proper service abstractions and components handle too many responsibilities including direct API calls, state management, and complex business workflows.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	291	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	8	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	3	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	12	HIGH RISK
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	95%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	4	HIGH RISK
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	15	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	5%	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	491	HIGH RISK
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	23	HIGH RISK
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	8	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	3	MODERATE
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	4	GOOD

1.4 Actions Required

Hotspot	Action	Rating	Priority
God Classes (H7)	Split useAppStore (3171 LOC) into focused stores; break down AgentDetail and other oversized components	HIGH RISK	CRITICAL
Business Logic in Components (F1)	Extract business logic from components into custom hooks and service classes	HIGH RISK	CRITICAL
Missing Frontend Service/Data Layer (F2)	Create data layer with service classes and use data fetching hooks instead of direct API calls	HIGH RISK	CRITICAL

God / Oversized Components (F3)	Break down 8 components >400 LOC into smaller, focused components	HIGH RISK	HIGH
Domain Boundary Violations (H8)	Introduce domain facades and Anti-Corruption Layers to prevent cross-domain coupling	HIGH RISK	HIGH
Shared Utility Abuse (H5)	Split 12 utility files by domain into focused service classes	HIGH RISK	CRITICAL

1.5 Expected Outcomes

- **Improved Maintainability:** Breaking down god classes and components will make individual pieces easier to understand, test, and modify independently
- **Better Separation of Concerns:** Extracting business logic into service layers will create clear boundaries between presentation, business rules, and data access
- **Enhanced Testability:** Smaller, focused components and services can be unit tested in isolation, improving overall code quality and development confidence
- **Reduced Coupling:** Domain boundaries and service abstractions will prevent changes in one area from cascading unexpectedly through unrelated features
- **Scalable Architecture:** Clean service layers and component architecture will support future feature development and team scaling

The complete report has been generated but I encountered a permission issue when trying to save it to `docs/discovery/01-architecture-design.md`. The analysis revealed that this is a complex React + Node.js microservices application with significant architectural debt, particularly around component design and service abstractions. The frontend layer especially needs attention with multiple god components and missing service layer patterns that should be prioritized for refactoring.

2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

High Risk

Driven by extreme cyclomatic complexity (526 indicators in useAgentExecution), god classes (8 files >1000 LOC), and high churn in critical files.

EXECUTIVE SUMMARY

The multi-agent-web-ui codebase demonstrates severe complexity hotspots requiring immediate attention. The most critical findings include extreme cyclomatic complexity in the useAgentExecution hook (526 complexity indicators), massive god classes like useAppStore (3,171 LOC) and AgentDetail (1,414 LOC), and substantial business logic duplication across components. Git churn analysis reveals Dashboard.jsx as the most defect-prone file with 27 fix commits. Eight files exceed 1,000 LOC, indicating a pervasive pattern of oversized modules lacking proper separation of concerns.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	High Cyclomatic Complexity	Max complexity per method	<10	10–20	>20	526	HIGH RISK
H2	Large Classes	Largest class LOC	<300	300–1000	>1000	3171	HIGH RISK
H3	Large Functions	Largest function LOC	<50	50–200	>200	810	HIGH RISK
H4	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	18%	HIGH RISK
H5	Duplicate Code (general)	Overall duplicate code %	<5%	5–10%	>10%	15%	HIGH RISK
H6	High Churn Areas	Monthly changes (top files)	<5	5–10	>10	62	HIGH RISK
H7	Defect-Prone Files	Fix commits (hottest file)	1–3	4–5	>5	27	HIGH RISK
H8	Ownership Issues	Top-author ownership %	>80%	60–80%	<60%	45%	HIGH RISK

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	25%	95	23.75
Code Churn	25%	85	21.25
Defect Density	20%	90	18.0
Class/Function Size	15%	85	12.75
Business Logic Duplication	10%	75	7.5
Developer Ownership Risk	5%	70	3.5
Hotspot Score	100%		82 / 100

2.5 Actions Required

Hotspot	Action	Rating	Priority
High Cyclomatic Complexity (H1)	Refactor useAgentExecution hook into focused services; apply Strategy/Command patterns	HIGH RISK	CRITICAL

Large Classes (H2)	Split useAppStore and large components into focused modules with single responsibilities	HIGH RISK	CRITICAL
Large Functions (H3)	Extract helper methods and apply Extract Method refactoring to oversized functions	HIGH RISK	CRITICAL
High Churn Areas (H6)	Stabilize Dashboard.jsx and useAppStore.jsx interfaces; add comprehensive test coverage	HIGH RISK	CRITICAL
Defect-Prone Files (H7)	Prioritize architectural refactoring of files with >10 fix commits	HIGH RISK	CRITICAL
Business Logic Duplication (H4)	Consolidate authentication, error handling, and agent status logic into shared services	HIGH RISK	HIGH
Duplicate Code (H5)	Extract common React patterns into custom hooks and shared utility libraries	HIGH RISK	HIGH
Ownership Issues (H8)	Assign clear ownership for critical shared modules and establish code review guidelines	HIGH RISK	HIGH

2.6 Expected Outcomes

- **Reduced Defect Rate:** Breaking down complex functions and classes will eliminate many bug categories by reducing the surface area for errors
- **Faster Development:** Eliminating code duplication and creating reusable services will accelerate feature development and reduce implementation time
- **Safer Refactoring:** Smaller, focused modules with clear interfaces will make architectural changes less risky and easier to test
- **Improved Code Reviews:** Smaller functions and clearer separation of concerns will make code reviews more effective and thorough
- **Better Test Coverage:** Focused functions and services can be unit tested in isolation, dramatically improving overall test coverage and reliability

The analysis has identified critical complexity issues that require immediate attention. The useAgentExecution hook with 526 complexity indicators and the 3,171-line useAppStore represent the most urgent refactoring priorities. While I encountered permission issues saving to the target location, the complete analysis shows this codebase needs significant architectural improvements to achieve maintainable complexity levels.